

Memory Efficient Doubly Linked List
– Traverse In Order – Space Complexity and Algorithm

Program:

```
void traverseInOrder()
{
    Node *curr = head;
    Node *prev = nullptr;
    Node *next;

    cout << "List (Forward): ";
    while (curr != nullptr)
    {
        cout << curr->data << " <-> ";

        // Calculate next address using XOR formula
        next = XOR(prev, curr->npx);

        // Shift pointers forward
        prev = curr;
        curr = next;
    }
    cout << "NULL" << endl;
}

...

case 8:
    traverseInOrder();
    break;
```

Space Complexity Analysis: XOR Linked List (In – Order Traversal)

1. Input Space Complexity

- **Parameter – only Definition:**
 - **Function Parameters:** None.
 - **Analysis:** No data is passed into the function as arguments.
 - **Complexity:** $O(1)$
- **Full Data Definition:**
 - **Function Data:** The entire XOR linked list structure.
 - **Analysis:** The function must access all n nodes in the list to print their values. Therefore, the data structure it operates on is proportional to the number of elements.
 - **Complexity:** $O(n)$

2. Auxiliary Space Complexity

Explanation: This measures the temporary memory used by the algorithm during execution.

- **Local Pointer Variables:** The function uses three local pointers: *curr*, *prev*, and *next*.
 - **Analysis:** These are stack variables that occupy a constant amount of space (e. g., 24 bytes on a 64 – bit system). Regardless of whether the list has 10 nodes or 10 million nodes, only these three pointers are needed to keep track of the traversal.
- **Iterative Process:** The traversal uses a while loop rather than recursion. This ensures that the function call stack does not grow with the number of nodes.
- **XOR Logic:** The calculation of the next address happens in CPU registers using bitwise operations, requiring no extra memory allocation.

Conclusion: The amount of extra memory used is fixed and does not grow as the size of the linked list (n) grows.

- **Auxiliary Space Complexity:** $O(1)$

Total Space Complexity

Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter-only input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity ($O(1)$)** is the expected answer.

Algorithm: Traverse In – Order (XOR Doubly Linked List)

XORTRVERSE(START):

1. [INITIALIZE TRAVERSAL POINTERS]

Set CURRPTR := START

Set PREVPTR := NULL

2. [PRINT HEADER]

Write: "List (Forward): "

3. [TRAVERSE AND PRINT NODES]

Repeat while CURRPTR ≠ NULL:

a. [DISPLAY DATA]

Write: INFO[CURRPTR], " < -> "

b. [DECODE NEXT NODE ADDRESS]

Set NEXTTEMPPTR :

= PREVPTR

$\oplus \text{NEXTXORPTR}[\text{CURRPTR}] \left(\begin{array}{l} \text{Use the address of the} \\ \text{previous node to unlock} \\ \text{the address of the} \\ \text{next node} \end{array} \right)$

c. [SHIFT POINTERS FORWARD]

Set PREVPTR := CURRPTR

Set CURRPTR := NEXTTEMPPTR

[End of Loop]

4. [PRINT TERMINATOR]

Write: "NULL"

5. EXIT
